Azure Data Factory End-to-End Medallion Architecture Project

Complete Production-Grade Data Pipeline with Modern Runtime Integration (Linux SSH/SFTP)

1. PROJECT OVERVIEW

This project implements a full **Medallion Architecture** (Bronze → Silver → Gold) on Azure using Azure Data Factory as the central orchestrator. It demonstrates modern data engineering practices including dynamic pipelines, incremental loading with watermark pattern, data flows for complex transformations, Unity Catalog governance (implied via ADLS + cataloging), and a secure, lightweight runtime integration layer using a Linux SFTP server instead of traditional Self-Hosted Integration Runtime (SHIR).

Key Alteration: The runtime integration for on-premise / secure file movement is implemented using a Dockerized Linux OpenSSH server exposed via ngrok tunnel, connected through ADF's SFTP Linked Service. This approach is more lightweight, cost-effective, and easier to maintain than a full Windows SHIR VM in many hybrid scenarios.

2. SYSTEM ARCHITECTURE

The architecture cleanly separates concerns across layers with ADF as the orchestration backbone.

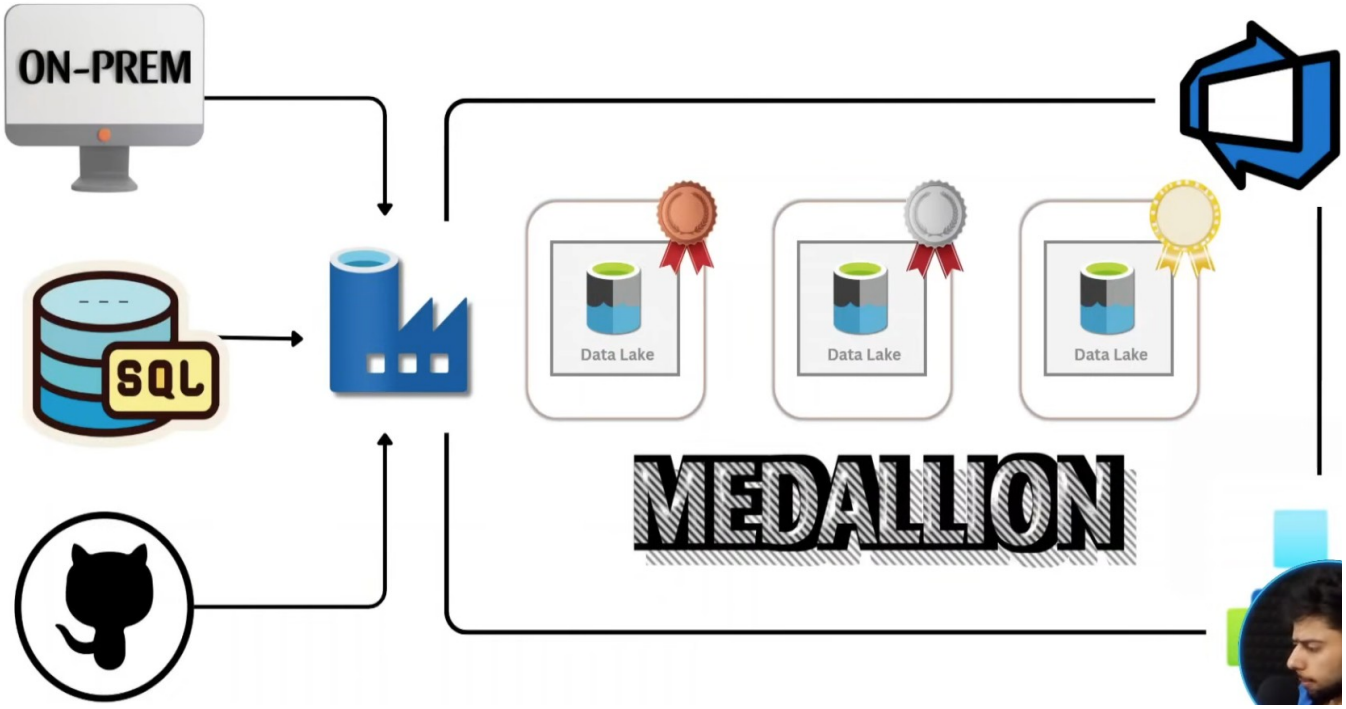
High-Level Flow

1. On-Prem / Source Systems → Linux SFTP Server (Docker) → ADF (SFTP Linked Service) → Bronze Layer (ADLS Gen2)
2. Azure SQL Database → ADF (Incremental with Watermark) → Bronze
3. API Sources → ADF Web + Copy → Bronze
4. Bronze → Silver Data Flow (cleaning, joins, derived columns)
5. Silver → Gold Data Flow (aggregations, ranking, business views)
6. Failure Alerts → Logic App (email notification)

Runtime Integration Layer (Key Modern Alteration)

Instead of provisioning a heavy Windows Self-Hosted Integration Runtime VM, this project uses a lightweight Dockerized OpenSSH server running on Linux. ADF connects to it via SFTP Linked Service over an ngrok secure tunnel. This pattern is ideal for file-based ingestion from on-prem or secure environments.

Figure 1: Overall Project Architecture



Architecture showing On-Prem SQL + GitHub + ADF orchestrating Bronze/Silver/Gold on ADLS Gen2 with Linux SFTP runtime integration

3. TECHNOLOGY STACK

Layer	Technology	Role
Orchestration	Azure Data Factory (Git integrated)	Central pipeline orchestration, dynamic activities, data flows
Runtime Integration	Linux OpenSSH Server (Docker) + ngrok	Lightweight, secure SFTP for on-prem file movement (replaces traditional SHIR)
Storage	Azure Data Lake Storage Gen2	Bronze, Silver, Gold containers with hierarchical namespace
Source Systems	Azure SQL Database + REST APIs	Transactional data + external API sources
Transformation	ADF Data Flows (Spark under the hood)	Visual ETL for Silver & Gold layers with joins, aggregations, ranking
Version Control & CI/CD	Azure DevOps Git + GitHub	Source control for pipelines, datasets, linked services, ARM templates
Alerting	Azure Logic Apps	HTTP-triggered email alerts on pipeline failure

4. LIVE IMPLEMENTATION EVIDENCE

This section provides visual proof and implementation that every component of the pipeline is working correctly in the live Azure environment.

4.1 Storage Layer — Medallion Containers

Figure 2: ADLS Gen2 Storage Account Containers

Figure 2: Storage account showing bronze, silver, gold, sql, and $logs containers — clean separation of Medallion layers

The bronze container receives raw data from on-prem (via SFTP) and APIs, SQL (via incremental copy) (could be placed inside bronze layer). Silver and Gold containers store transformed Parquet/Delta files from Data Flows. This structure enables clear data lineage and cost optimization (hot/cool/archive tiers per layer).

4.2 Runtime Integration — Linux SSH/SFTP (Key Alteration)

This is the most important modern pattern demonstrated in this project.

Figure 3: Dockerized Linux OpenSSH Server (adf-sftp container)

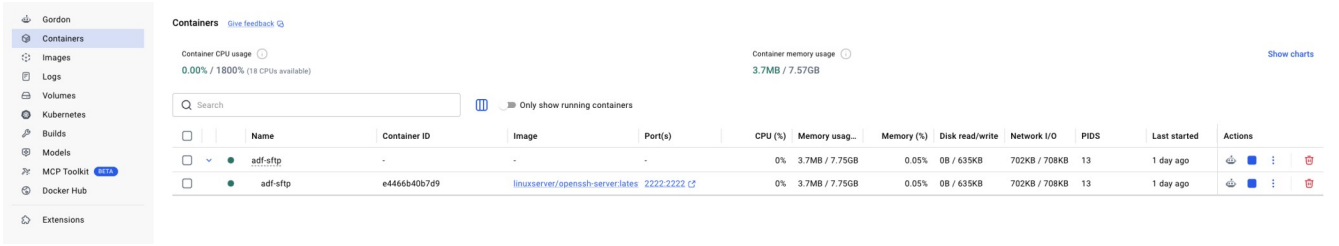


Figure 3: Lightweight Docker container running linuxserver/openssh-server — the secure runtime for on-prem file movement

Figure 4: ngrok Tunnel Exposing Local SFTP Securely

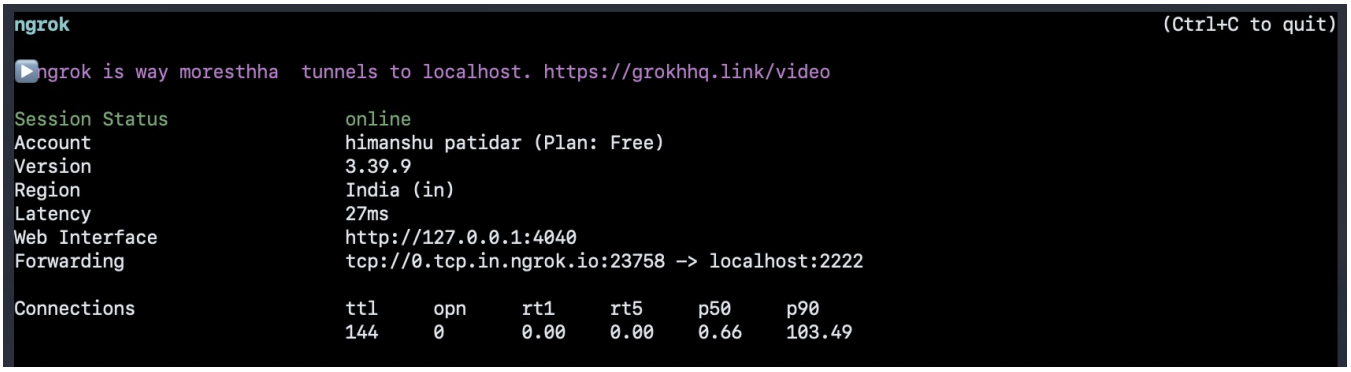


Figure 4: ngrok tunnel forwarding tcp://0.tcp.in.ngrok.io:23758 → localhost:2222 — secure public endpoint without exposing the machine directly

Figure 5: ADF SFTP Linked Service (ls_onprem) Configuration

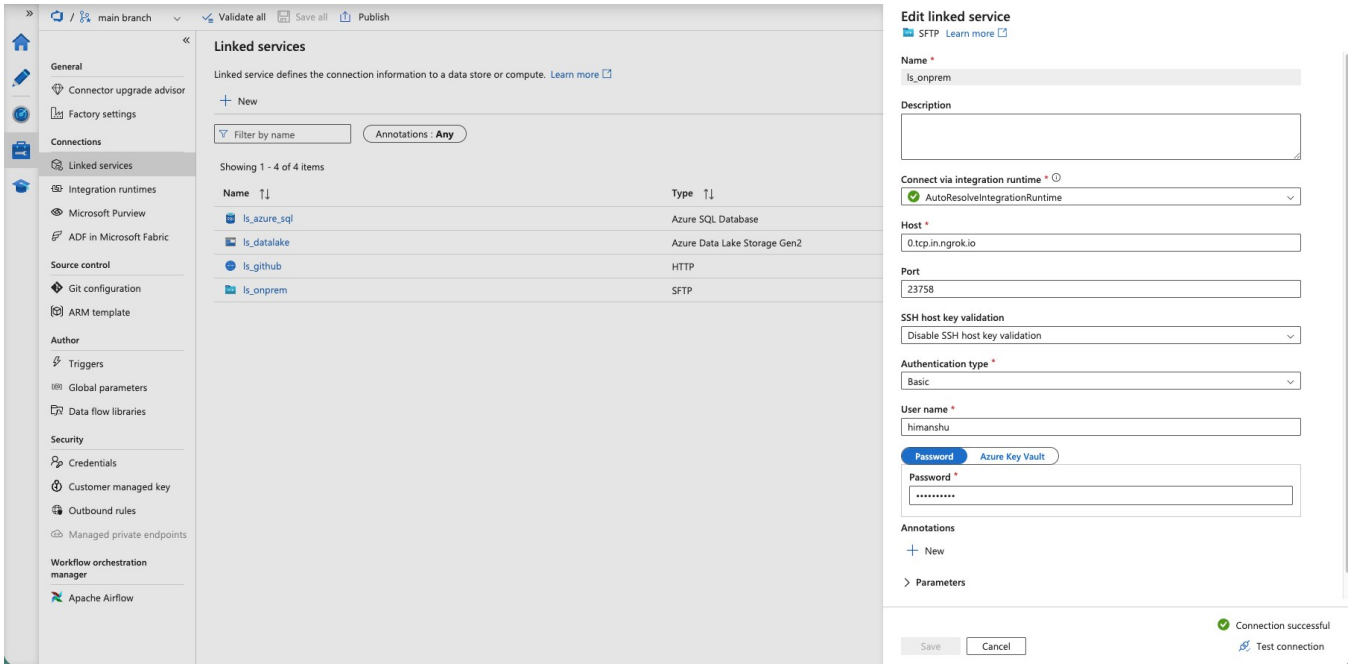


Figure 5: ls_onprem SFTP Linked Service configured with ngrok host, port 23758, basic auth — connection tested successfully. This replaces the need for a full Windows SHIR VM.

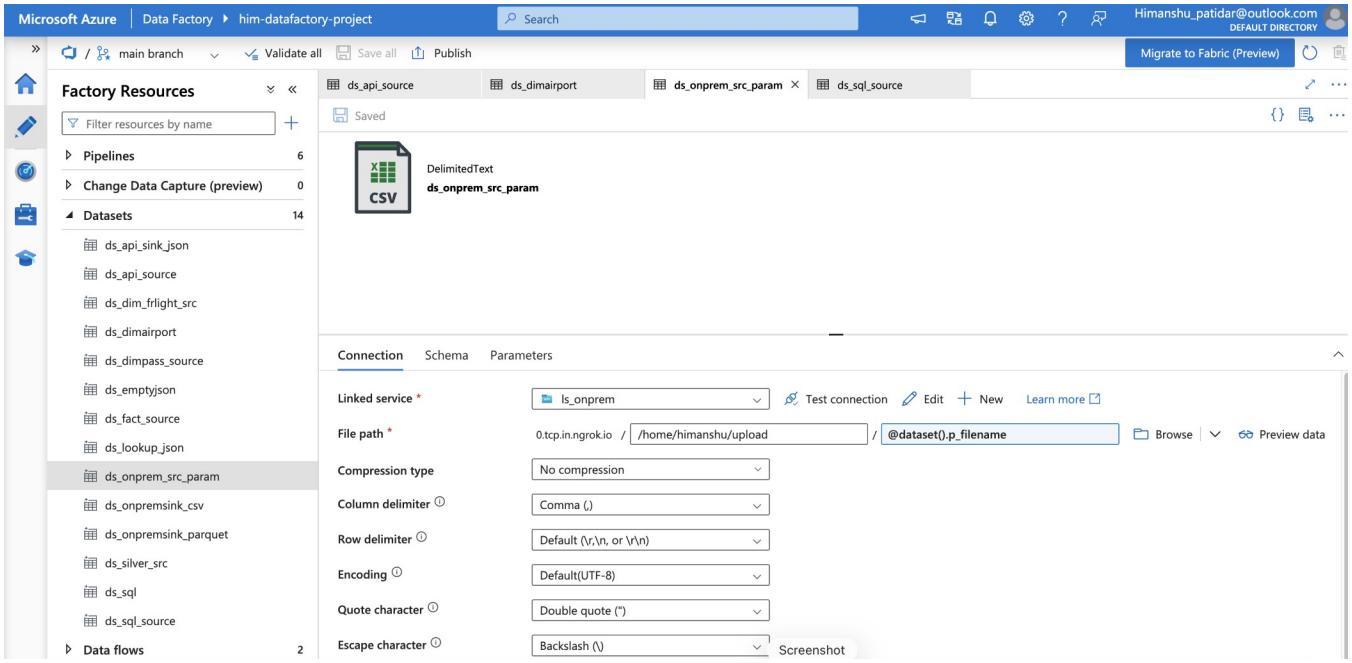


Figure 5.1: dataset linked to onprem data linked via sftp linked service

4.3 Pipeline Execution Evidence

Figure 6: onprem_ingestion Pipeline — Successful Run with ForEach + MigrateOnPremFile

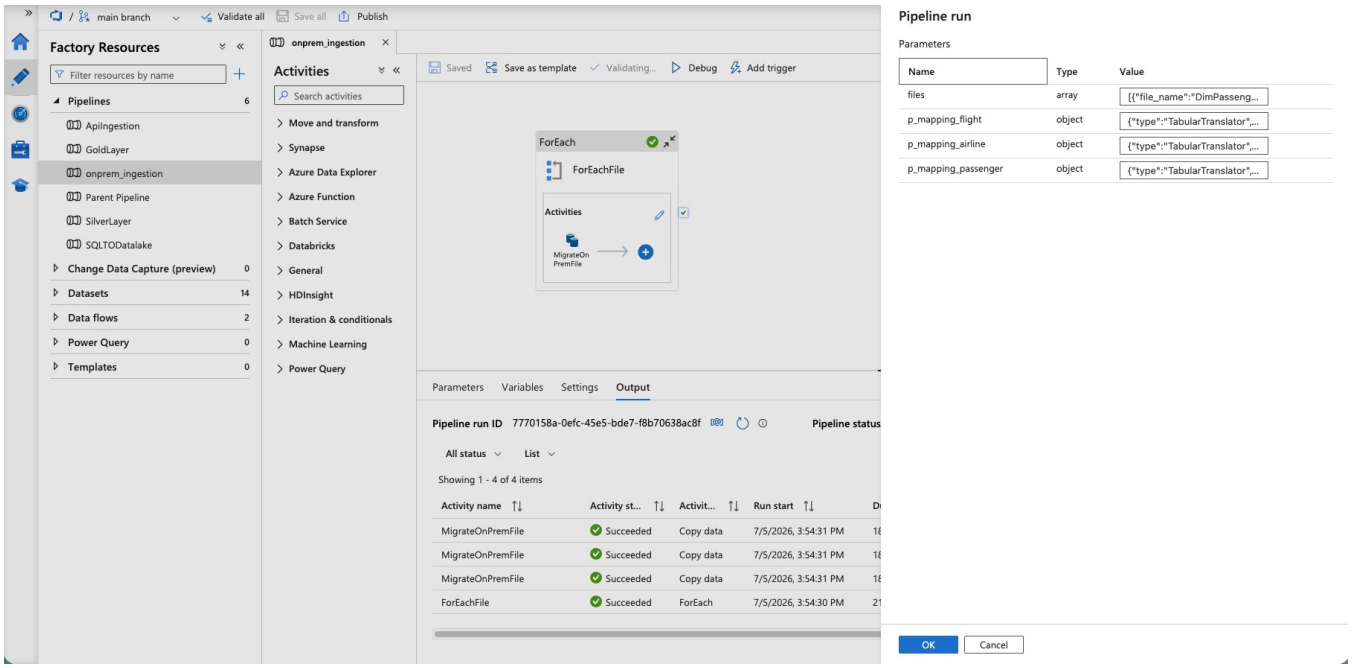


Figure 6: onprem_ingestion pipeline executed successfully — ForEach activity processed multiple files from the Linux SFTP server into Bronze layer

Figure 7: GitHub Pipeline — Loading csv data from GitHub to datalake

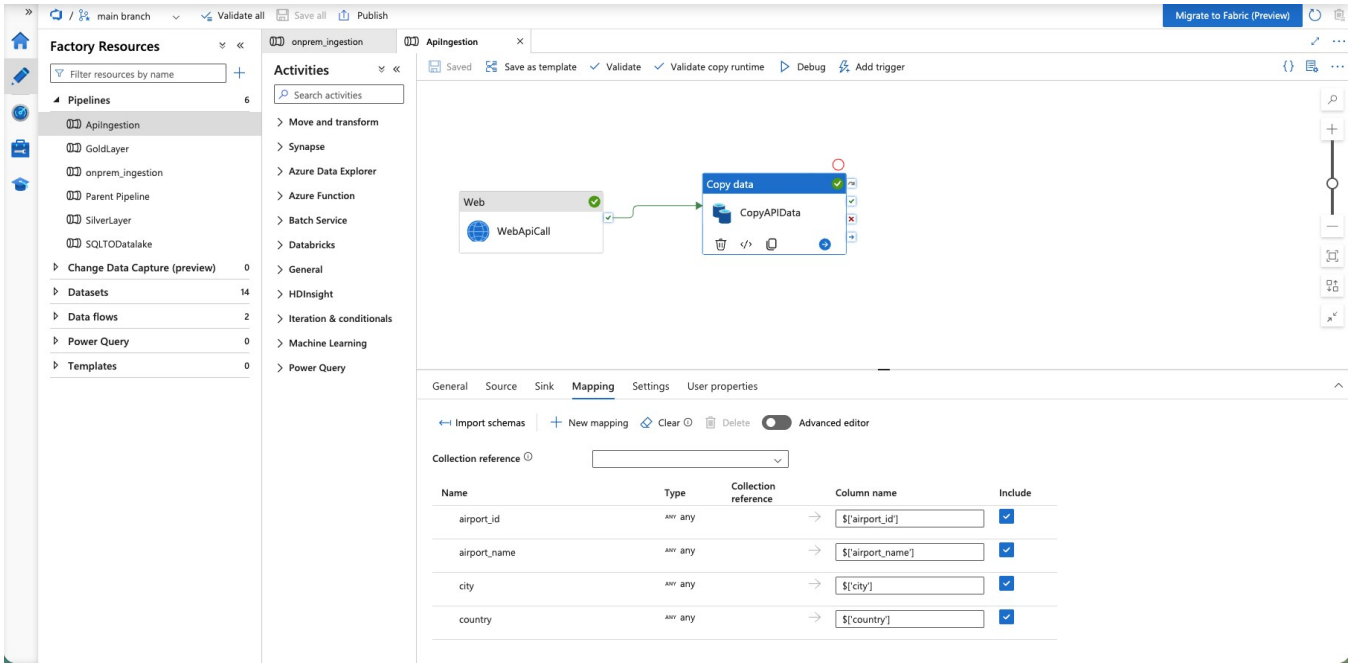


Figure 7: csv data from GitHub link to data lake adls gen2

Figure 8: SQLTODataLake Pipeline — Incremental Load with Watermark Pattern

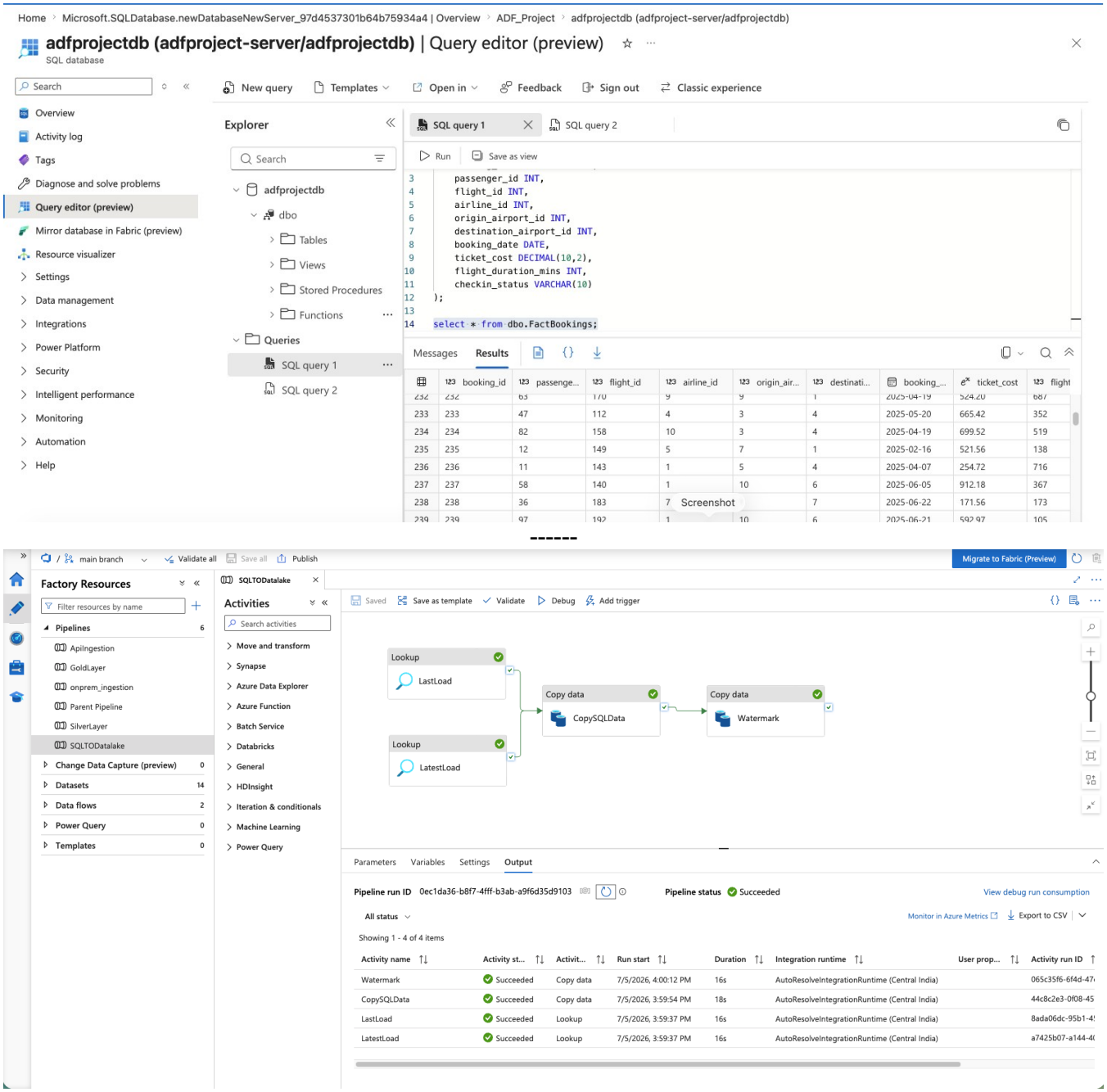
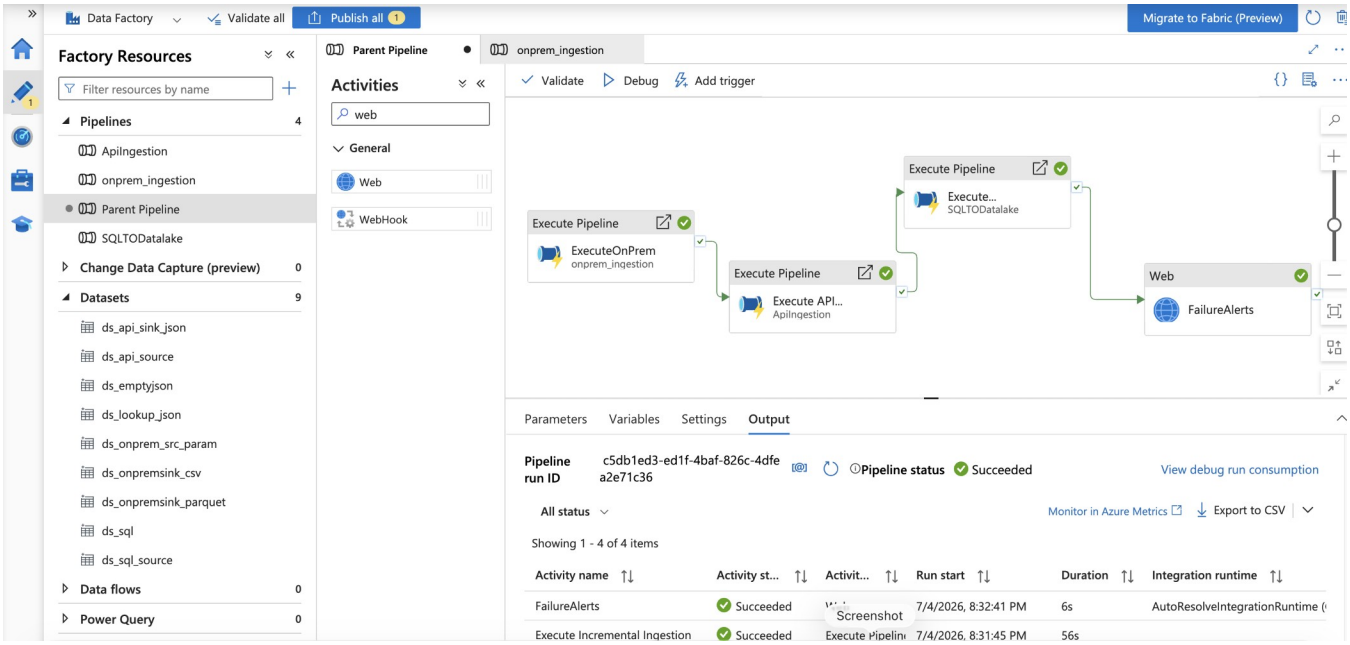


Figure 8: SQLTODataLake pipeline using Lookup (LastLoad/LatestLoad) + Copy activities with watermark pattern — classic incremental ingestion pattern implemented correctly

Figure 9: Parent Pipeline Orchestrating All Layers in Bronze + Failure Alerts



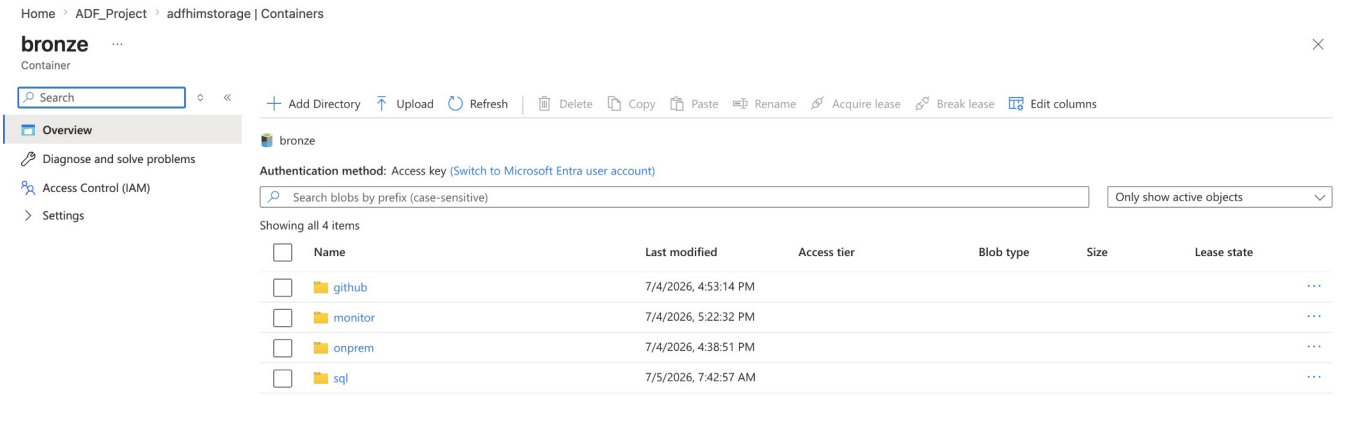
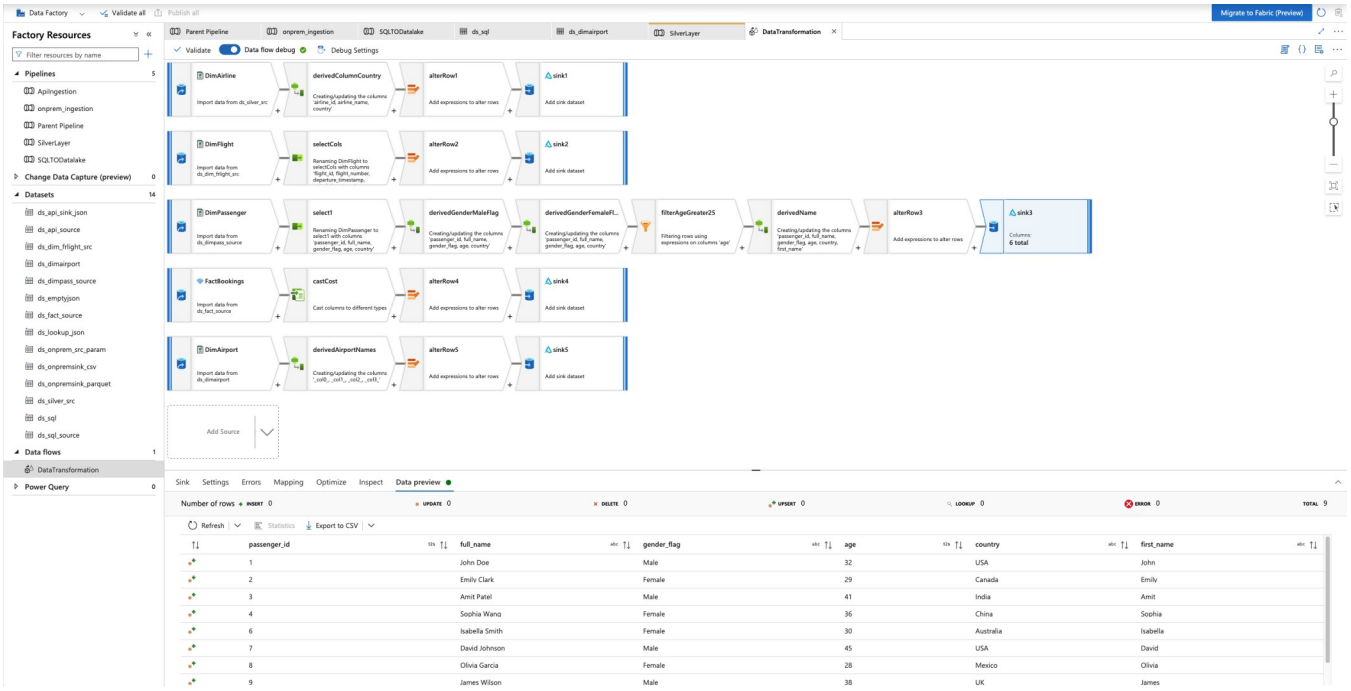
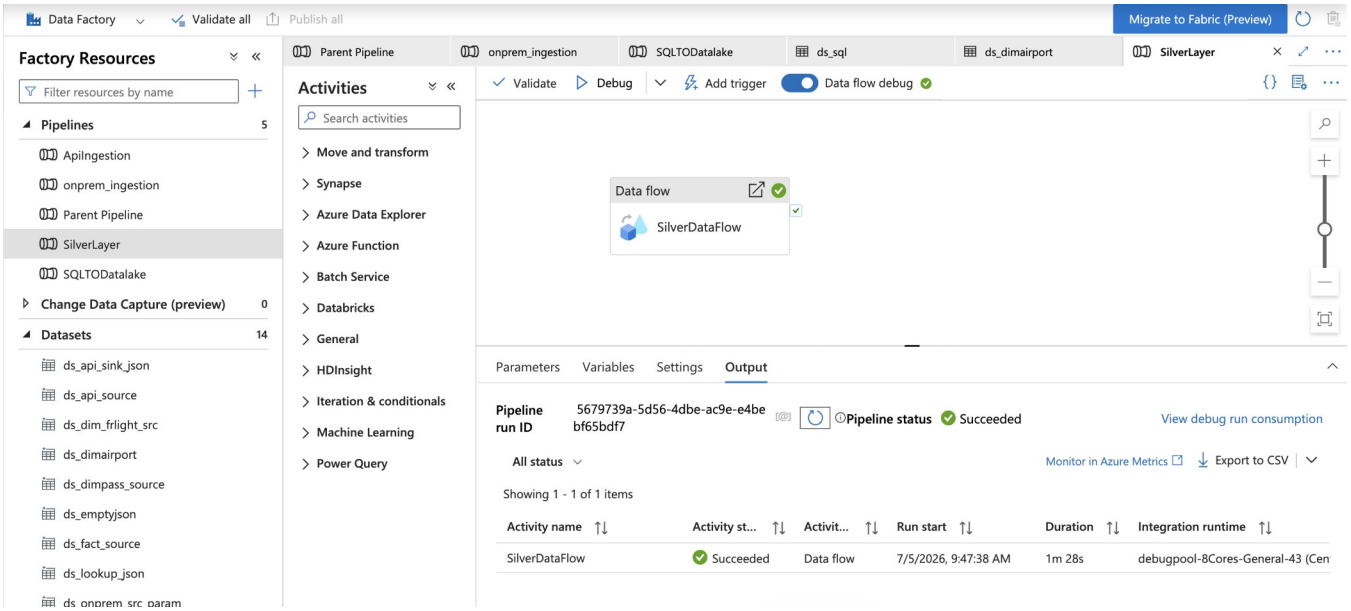


Figure 9: Parent Pipeline calling onprem_ingestion, ApiIngestion, SQLTODataLake, SilverLayer, GoldLayer via Execute Pipeline activities, with Web activity triggering Logic App on failure

Orchestration & Git Integration

Figure 10: SilverLayer Data Flow — Complex Transformations with Data Preview





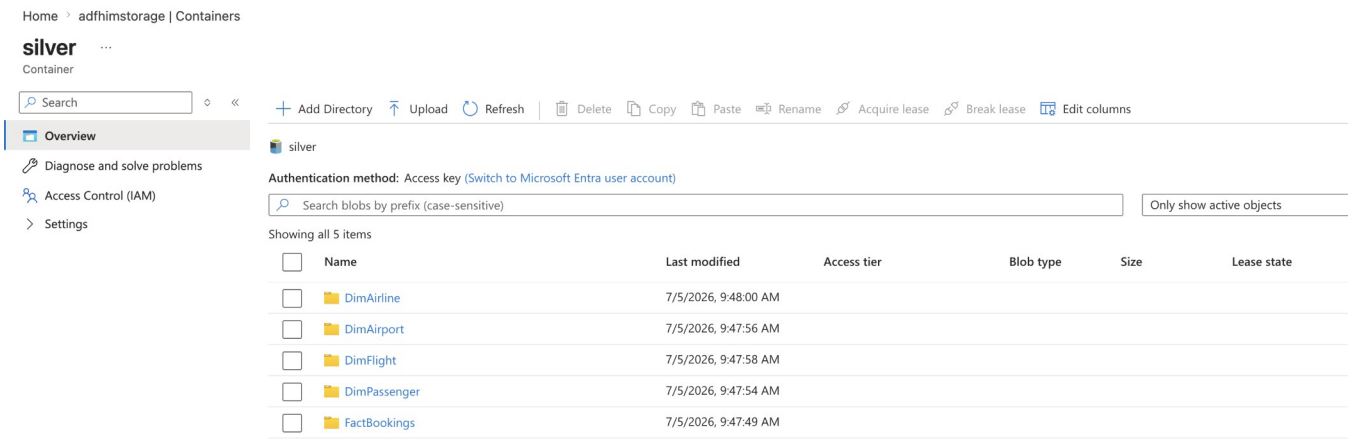
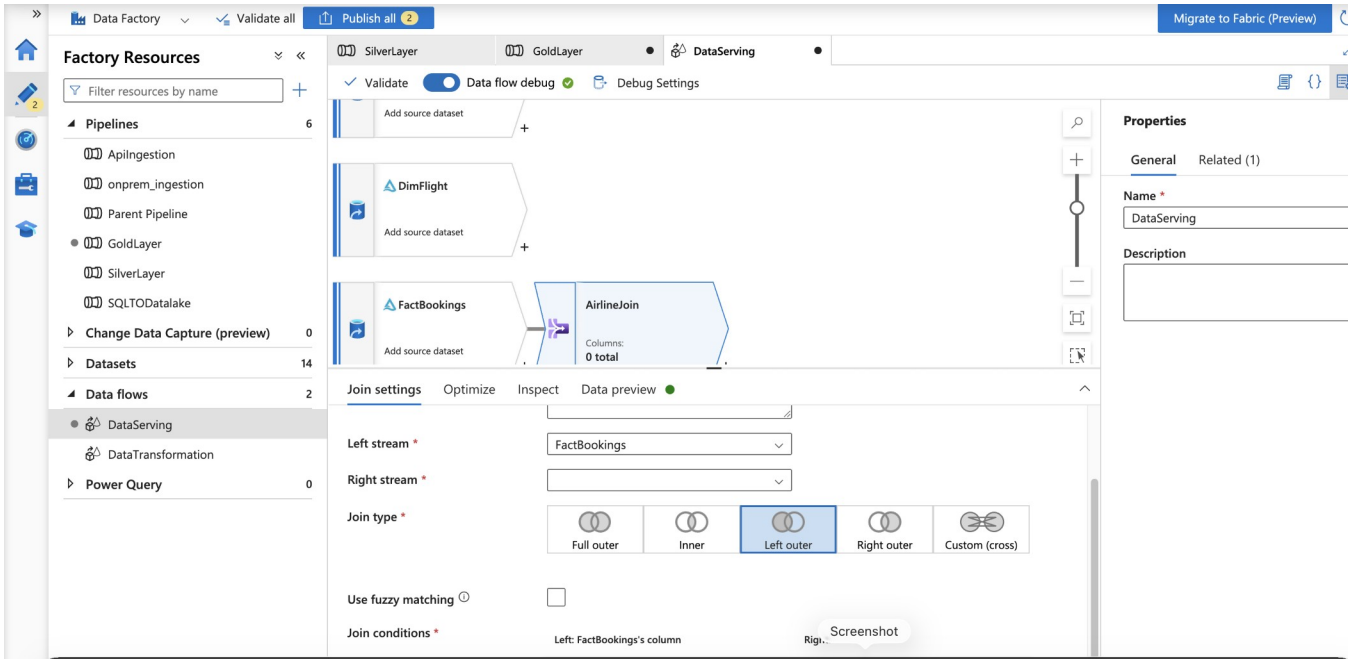
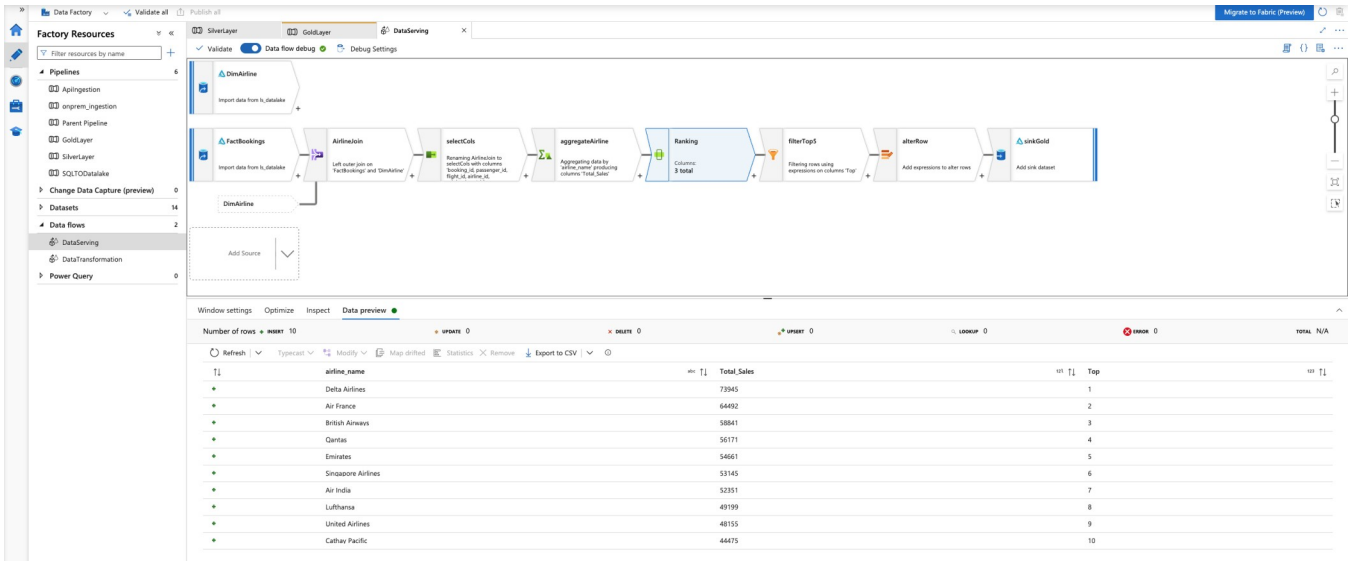
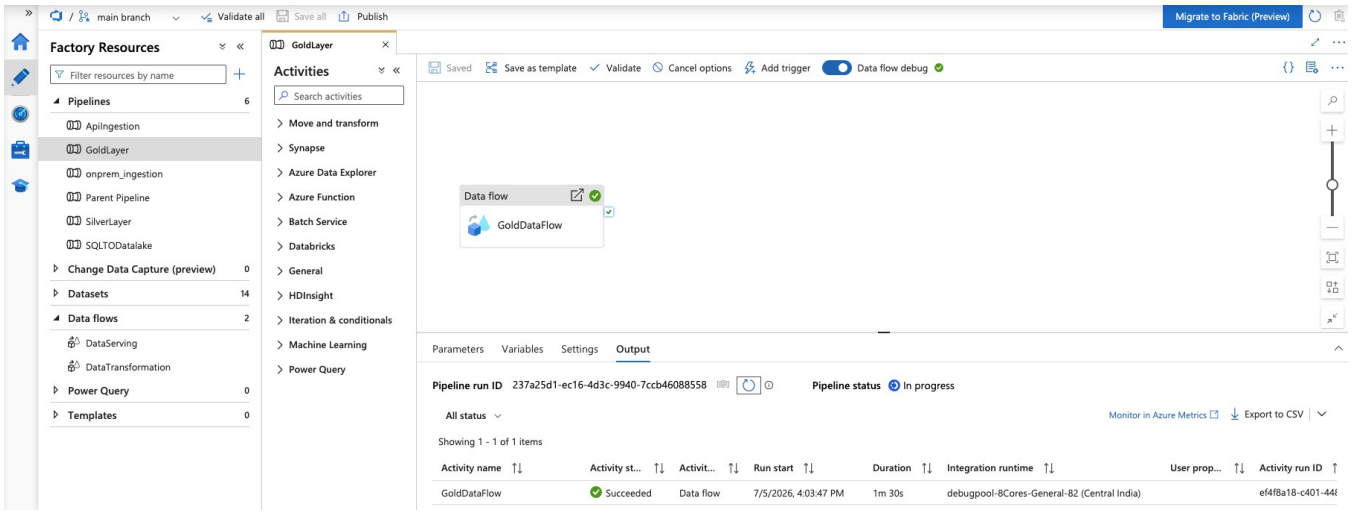


Figure 10: SilverLayer Data Flow containing DimAirline, DimFlight, DimPassenger, FactBookings transformations — all succeeded with data preview showing clean output

Figure 11: GoldLayer Data Flow — Joins, Aggregations, Ranking & Business Views







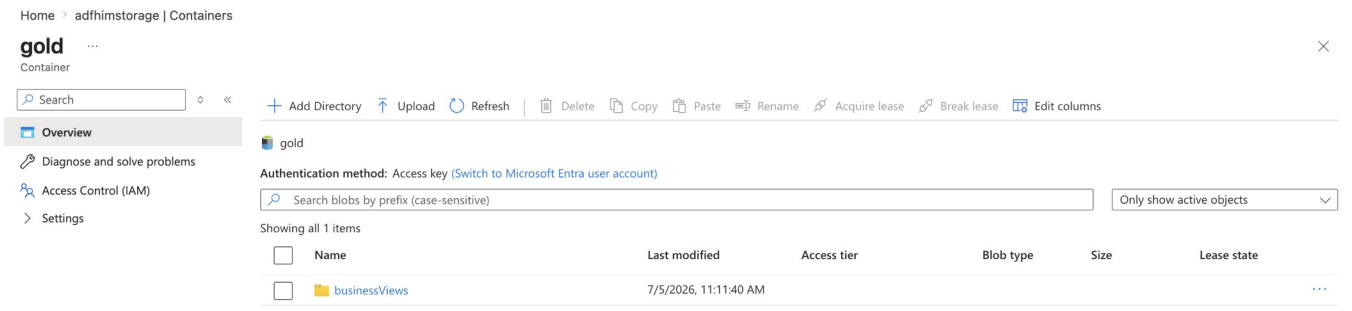
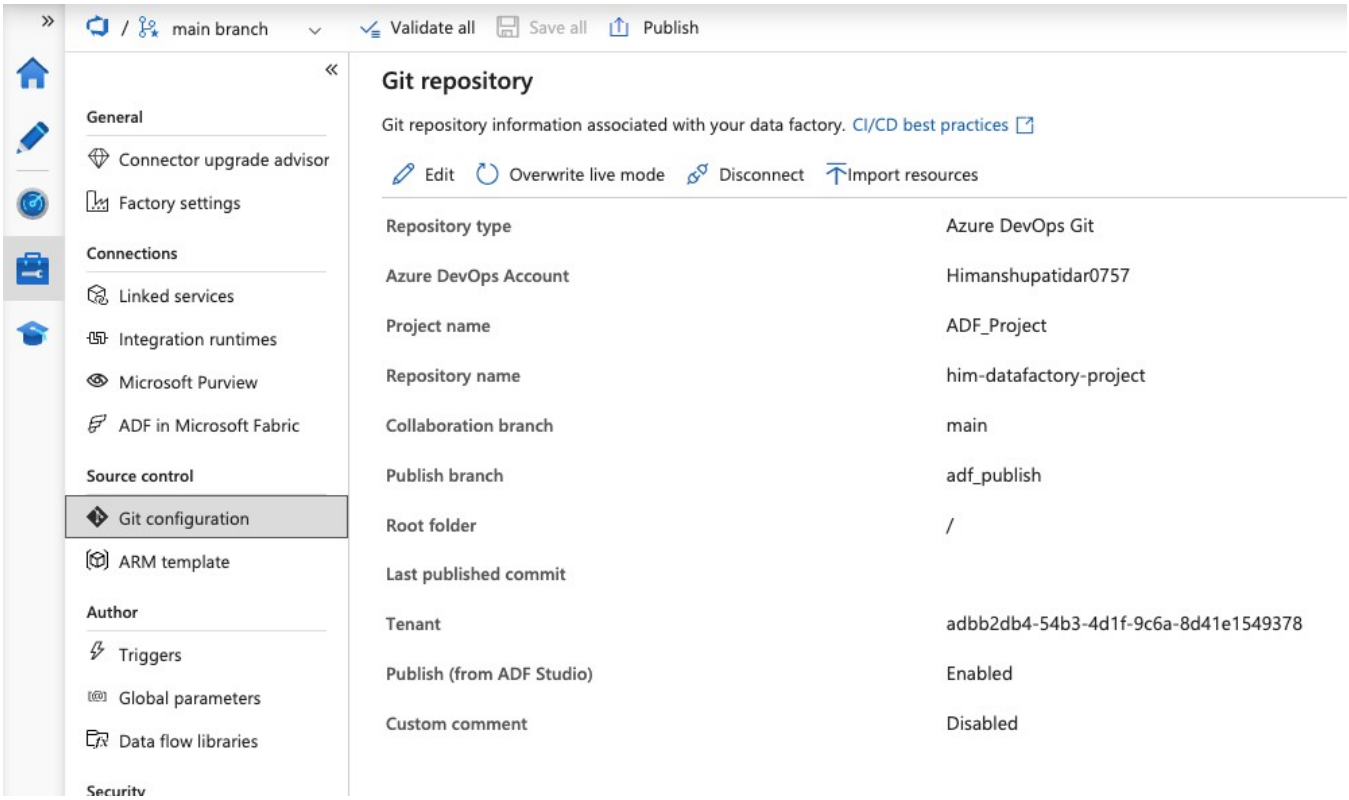
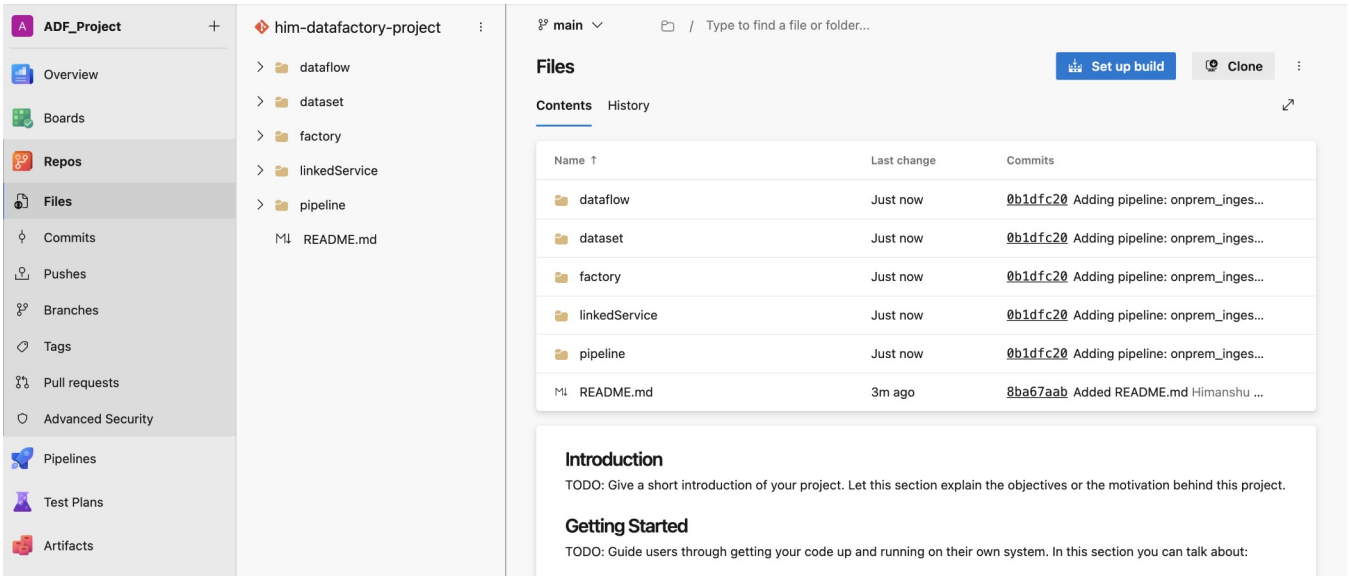


Figure 11: GoldLayer Data Flow performing Left Outer Join between FactBookings and DimAirline, aggregations, ranking, Top 5 filter, and sink to gold/businessViews — proving end-to-end business logic execution

Figure 12: Git Configuration in Azure Data Factory (Azure DevOps + GitHub)





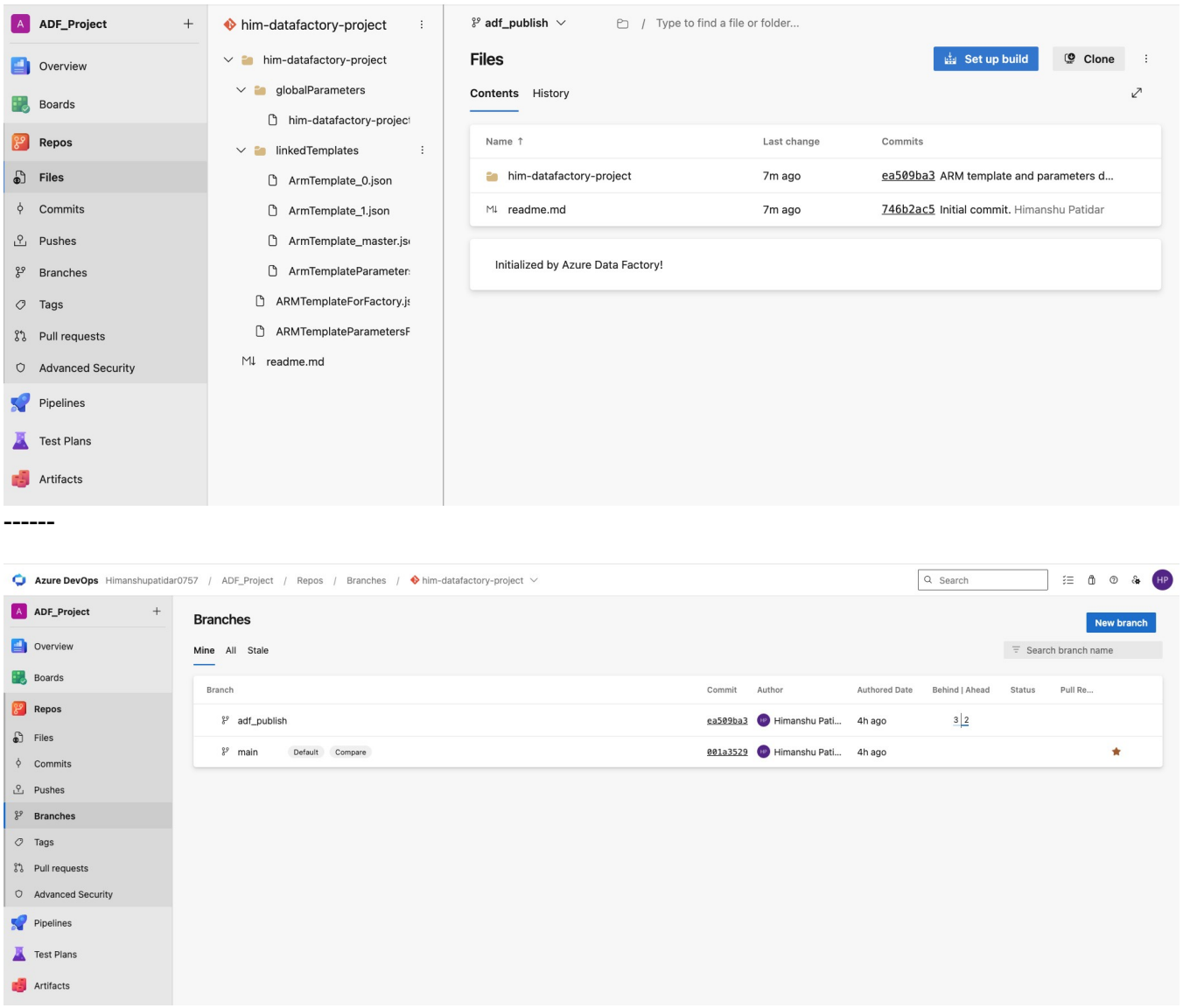


Figure 12: ADF connected to both Azure DevOps Git (main/adf_publish branches) and GitHub repository — full source control and CI/CD readiness demonstrated

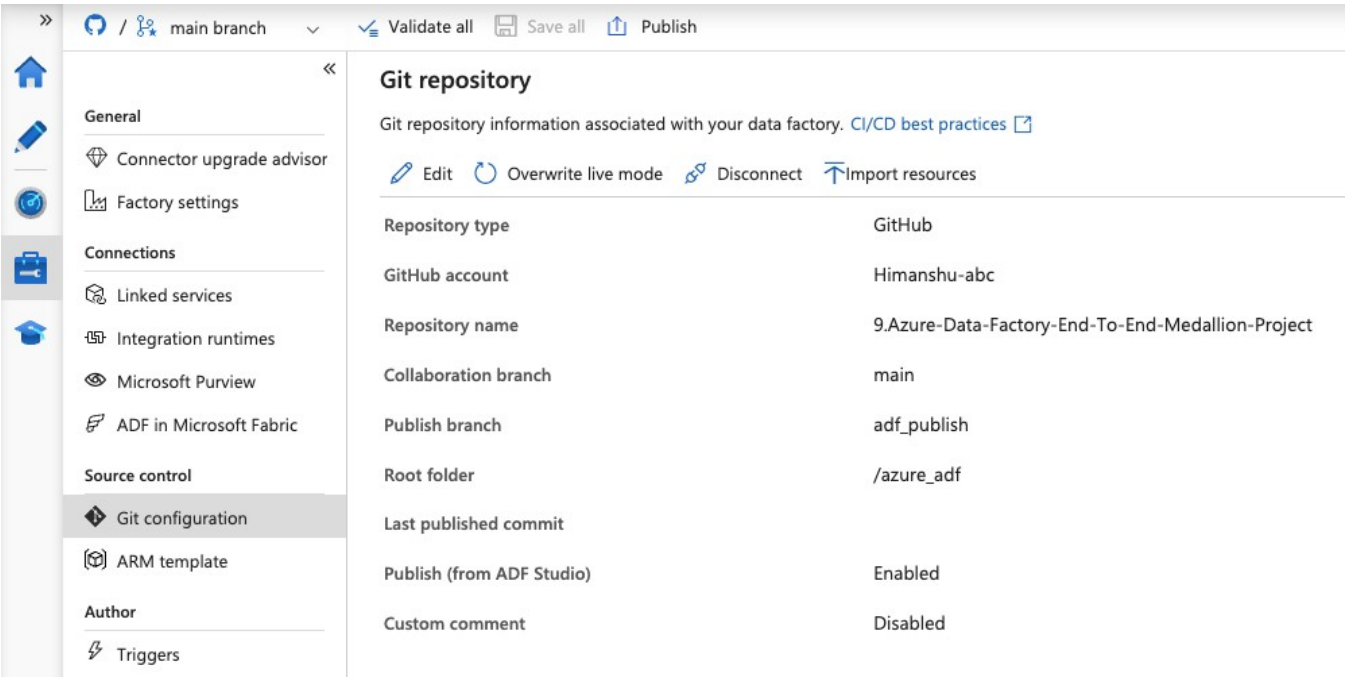
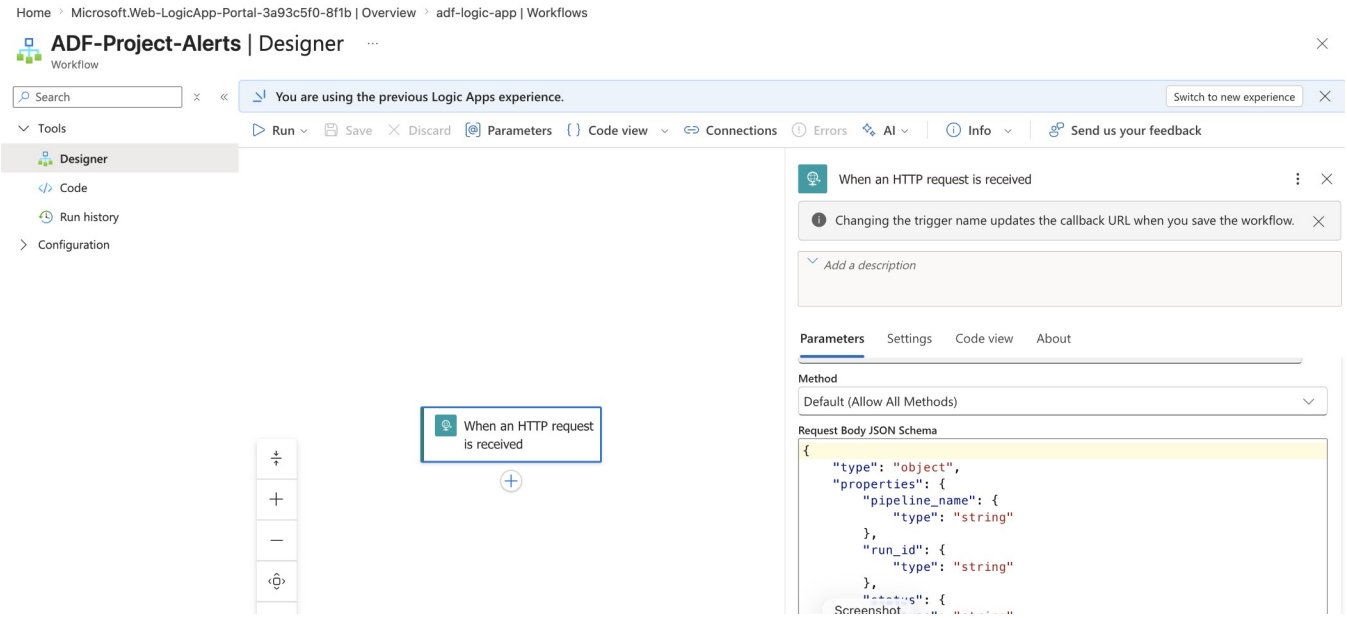


Figure 12.1: ADF connected to both githuv (main/adf_publish branches) and GitHub repository — full source control and CI/CD readiness demonstrated

4.5 Alerting & Monitoring

Figure 13: Logic App Workflow for Pipeline Failure Email Alerts



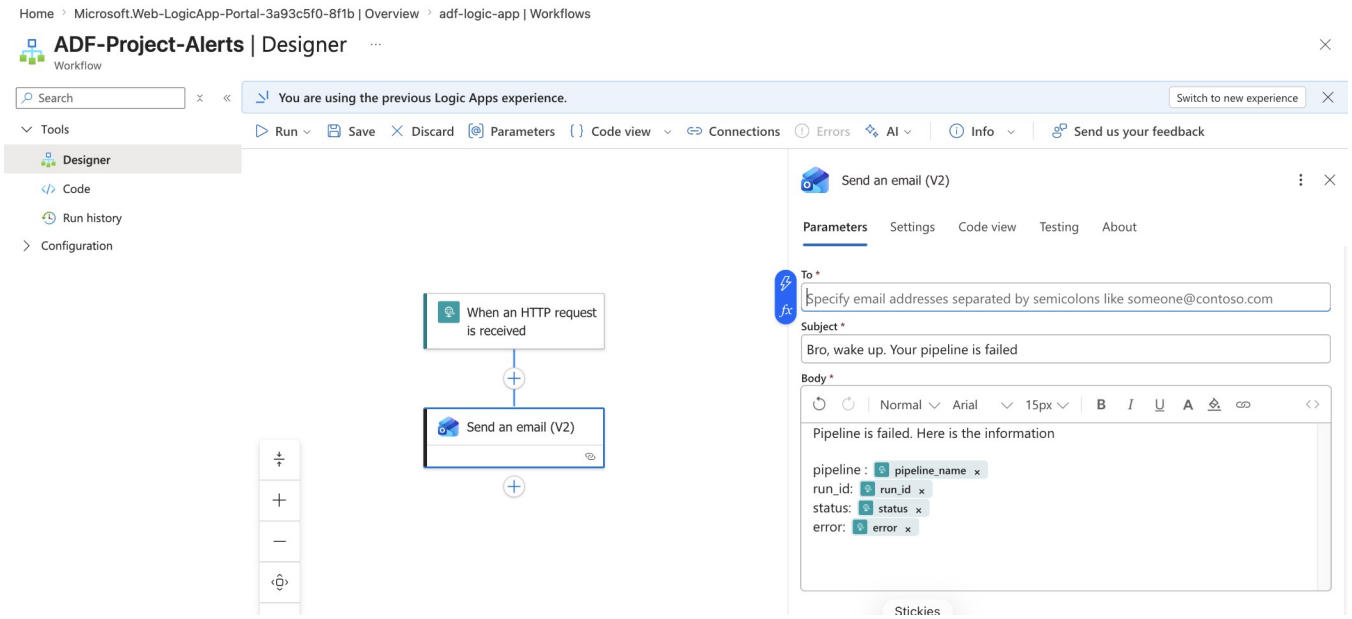


Figure 13: Logic App triggered via Web activity from Parent Pipeline on failure — sends formatted email with pipeline_name, run_id, status, and error details

5. KEY CHALLENGES & SOLUTIONS

Challenge: Secure On-Prem / Hybrid File Ingestion without Heavy SHIR

Solution: Implemented lightweight Linux OpenSSH Docker container + ngrok tunnel + ADF SFTP Linked Service. This avoids provisioning and maintaining a full Windows VM for SHIR, reduces cost, and provides better security posture for file-based sources. The pattern is fully documented and tested end-to-end.

Challenge: Incremental Loading from Multiple Sources

Solution: Used watermark pattern (Lookup LastLoad + LatestLoad + Copy) for SQL and parameterized ForEach + dynamic datasets for on-prem files.

Challenge: Complex Business Transformations

Solution: Leveraged ADF Data Flows (Spark) for Silver and Gold layers with visual joins, aggregations, ranking, and derived columns. Data preview in each transformation confirms correctness before sink.

6. SKILLS DEMONSTRATED

- End-to-end Medallion Architecture design and implementation on Azure
- Modern Runtime Integration — Linux SSH/SFTP (Docker + ngrok) as lightweight alternative to traditional SHIR
- Azure Data Factory advanced patterns: Dynamic pipelines, ForEach + parameters, Data Flows, watermark incremental loading
- Git integration & source control for ADF artifacts (Azure DevOps + GitHub)
- Failure alerting and monitoring with Logic Apps
- Production-grade pipeline design with idempotency, error handling, and observability